# Progress Report 2

Private and Verifiable Ethereum State Reads

CS6666 | Blockchain and Distributed Systems

**Rohan Arora** | Roll No. **CS26S022**

## 1. Work Since Report 1

Report 1 covered the WETH workload, the initial Rust code and the checks on RPC providers. Since then, the work has moved to collecting a set of real proofs, checking them locally and retrieving them through PIR. The repository now contains the code, retained inputs and experiment results. Helios has also been selected for the light client part.

The main progress is that a proof record can now pass through the local PIR code and then through the Ethereum proof verifier. The state root used in this test still comes from the provider. Connecting it to live Ethereum consensus is work that remains.

## 2. Collecting and Checking Real Proofs

The new corpus contains 32 addresses at Sepolia block **11919648**, using the same WETH9 contract as before. Each request uses the exact block hash through EIP-1898. The corpus was reproduced using a second provider. One later request failed because a provider no longer had the old state. That failure was retained, and the collector did not silently move to another block.

Two Rust libraries, `alloy-trie` and `eth_trie`, were tested on these proofs. Both checked all 32 records and passed 313 rejection checks per candidate. The current verifier uses **eth_trie 0.6.1**. It can recover the account data from a proof, which lets the code obtain the storage root without trusting the account fields supplied separately by the RPC provider.

| Check | Result |
|---|---|
| Real proof corpus | 32 records collected at one fixed block hash |
| Balance results | 24 nonzero inclusions and 8 proven absences under the supplied root |
| Verifier checks | Account and storage verification, expected key and snapshot checks, proof size limits and explicit errors |
| Rust workspace tests | 27 passing tests, including incorrect values, malformed nodes and missing proof paths |

The checking order is important. First, the account proof gives the WETH storage root. The storage proof is then checked against that root for the requested key. If a required node is missing, the result is an error. It must not be treated as a zero balance. The eight zero cases above have valid absence proofs under the supplied root.

The verifier is kept separate from PIR and the light client. It only receives the expected request details and proof data. This means the retrieval method can change without rewriting Ethereum proof verification. It also makes it easier to test which check rejects a bad response.

## 3. Choosing Helios for the Light Client

Helios has been selected to check where the execution state root comes from. So far, the test uses its consensus code with a retained synthetic bootstrap and five signed Deneb updates. The adapter returns the finalised execution block number, block hash and state root. Ten Rust tests and four source integrity tests check this part of the experiment.

One useful case separates the optimistic head from the finalised head. The optimistic head moves to slot 96, while the finalised head stays at slot 72. The adapter continues using slot 72 until a verified update advances finality. Waiting longer does not cause it to accept the optimistic head as finalised.

This is still a fixture test with a 32 member test committee. It does not authenticate the WETH corpus on Sepolia. The remaining work includes live sync, a trusted recent checkpoint, support for the required fork and restart recovery. How much of Helios's existing sync code to reuse is also still open.

## 4. How the Proof Records Are Stored

The PIR tests use fixed size records, while Ethereum proofs have different sizes. A binary encoder and decoder were added to try out padded proof records. All 32 records were encoded, decoded and verified again. The decoder also rejects oversized records and malformed frames.

| Format | Result on the current corpus |
|---|---|
| Compact JSON | 4,599 to 5,545 bytes per record |
| Binary before padding | 2,254 to 2,731 bytes per record |
| 3,072 byte records | All 32 fit, with 18.05% of database bytes used for padding |
| 4,096 byte records | All 32 fit, with 38.54% padding |

A 3 KiB record is the smallest size tested that holds this whole corpus. It is not a general limit for Ethereum proofs. At 2,560 bytes, ten records do not fit. A larger corpus or another snapshot may therefore need a different size. The code rejects such a record rather than removing proof nodes to make it fit.

There is also repeated data in the proofs. The WETH account proof is common to all requests, so it is stored once in public metadata. The storage proofs have 192 node references but only 142 distinct nodes. Removing duplicates could save 35.84% of the raw storage proof bytes, before indexing and padding. Whether this helps PIR is still a separate question, since retrieving several nodes may require more private queries than retrieving one complete proof.

## 5. PIR Experiments

The local PIR tests use a fixed version of IKPIR. They compare direct index retrieval with full keyword lookup, using both SimplePIR and FrodoPIR. Each combination was tested with 3,072 and 4,096 byte records. All eight configurations recovered and verified the 32 records, giving **256 successful local PIR reads**.

| Engine | Retrieval mode | Setup minimum (bytes) | Query + reply (bytes) |
|---|---|---|---|
| SimplePIR | Direct index | 15,667,284 | 12,504 |
| FrodoPIR | Direct index | 19,243,072 | 12,504 |
| SimplePIR | Keyword | 125,664,173 | 98,720 |
| FrodoPIR | Keyword | 154,345,093 | 98,720 |

These figures use 3,072 byte records. Query and reply sizes include the message framing. The setup sizes are minimum estimates from the setup data, not actual network downloads. They also exclude the common manifest and index map. The client needs setup information, including a hint used to decode replies, so counting only query and reply bytes would leave out a significant cost.

Direct indexing costs less than keyword lookup in this test. The client finds the record index using a public map, whereas keyword lookup retrieves buckets from multiple segments. Direct indexing is therefore the next baseline to investigate. This result alone does not settle the final backend choice.

PIR hides which record is requested, but it does not make the returned bytes trustworthy. The client must still derive the expected storage key, check the snapshot and verify the proof. A failed private request must not trigger an ordinary RPC lookup for the same address.

The full padded generation, metadata and index map together occupy 109,390 bytes. That is a very small dataset for judging PIR performance. These runs show that the pieces work together. They do not show how useful PIR will be at larger scales. They also run locally, so network delay and concurrent clients have not yet been measured.

## 6. What the Project Is Trying to Find Out

The useful question now is **when private, verifiable Ethereum reads become practical**. Getting a PIR query to work is necessary, but it is only the starting point. The results should help explain which workloads justify the extra setup and server work, and which costs make the approach less useful.

The current 32 record corpus is mainly a correctness test. Its comparison with downloading the full dataset cannot answer the scale question. At the same time, increasing the number of records is not enough to claim that PIR is better. The comparison needs to include how often the client reads, how much it can store and how often the snapshot changes.

I want the final evaluation to give a concrete answer for a stated workload, rather than just show that one query succeeded. For example, it should explain how the costs change between a client making one read and a client making many reads before the next snapshot.

## 7. Plan for the Next Experiments

The existing real proofs will stay in the regression tests. The scale experiments will increase the number of records and total database size across several orders of magnitude. The sizes need to be chosen after estimating memory use. Security parameters will stay fixed, rather than being weakened to make a larger test run.

| What will vary | What needs to be measured |
|---|---|
| Proof representation | Complete proof records versus shared trie nodes, including padding, index maps and the number of private queries |
| Client usage | First setup, repeated reads and moving to a new snapshot, including hint costs and client storage |
| Server resources | Preprocessing time, query time, memory use and latency distributions, including sizes that cannot be run |
| Comparison methods | Downloading and caching the generation once, and ordinary RPC as a revealing reference |

Synthetic data can help measure backend scaling before a larger real corpus is available. Simply repeating these 32 records would not tell us how real proof sizes, shared nodes or updates behave. Those results will be labelled separately. Network measurements will also be kept separate from local compute timings.

The node based approach needs a privacy check as well as a size comparison. If the number or sequence of node requests reveals part of the selected path, smaller storage is not enough. Each comparison must use the same requested data and proof requirements.

## 8. Work Remaining

The next step is the scale and representation comparison. After that, the record layout and PIR baseline can be reviewed before building snapshot publication and the complete client. Helios still needs live sync and recovery tests. A new corpus must then be tied to a root accepted by the light client, or an authenticated history path is needed for an older block.

The project is still in Phase 1. The verifier and Helios choices are settled. The final PIR backend, record size, Helios integration method and repository licence are not. Network serving and checks that all files belong to the same published generation also remain to be implemented.

### References used


1. **eth_trie 0.6.1.** Account and storage proof verification in the Rust adapter. Source
2. **IKPIR.** Implementation used for the direct and keyword PIR experiments. Source
3. **Helios.** Consensus code used for the signed update and finalised root tests. Source
4. **SimplePIR and DoublePIR.** Background on setup and communication costs, USENIX Security 2023. Source
5. **EIP-1186.** The account and storage proof response format. Source